

Курс лекций: «Введение»

Как читать эти лекции: каждый термин сначала даётся точным определением (📖), затем объясняется простыми словами (🗨️) и, где возможно, через аналогию из жизни (💡). Незнакомые слова ищите в словарике в начале лекции.

Лекция 1. История возникновения языка C#

Словарик к лекции

Термин	Что означает
Язык программирования	Формальный язык, на котором человек записывает инструкции для компьютера
Платформа	Среда, в которой выполняется программа: она предоставляет готовые библиотеки и «движок» исполнения
Компилятор	Программа-переводчик: превращает исходный код человека в код, понятный компьютеру
Исходный код	Текст программы, написанный программистом (файлы <code>.cs</code> для C#)

1.1. Что нужно знать перед историей: C# — это язык, .NET — это платформа

Новички часто путают эти два понятия, поэтому зафиксируем сразу:

- **C#** — язык программирования. Это *правила записи* программ (как английский язык — правила, по которым пишут книги).
- **.NET** — платформа. Это *среда исполнения*: библиотека готовых классов (как словарь готовых слов) + «движок», который выполняет скомпилированную программу.

💡 **Аналогия:** C# — это язык, на котором вы пишете письмо, а .NET — это почтовая служба, конверты и правила доставки, благодаря которым письмо дойдёт до адресата на любом континенте.

1.2. Предыстория: почему Microsoft понадобился новый язык (1995–1999)

В конце 1990-х Microsoft оказалась под давлением трёх факторов.

Фактор 1. Java стала угрозой. В 1995 году компания Sun Microsystems выпустила язык **Java**. Его главная особенность: программа компилировалась не в машинный код конкретного процессора, а в **промежуточный байт-код**, который выполнялся специальной программой — **виртуальной машиной (JVM)**. Это позволяло запускать одну и ту же программу на разных системах (Windows, Linux, Solaris) — отсюда лозунг «*Write once, run anywhere*» («напиши один раз — запускай где угодно»). Java быстро завоёвывала корпоративный рынок.

Фактор 2. C++ был мощным, но опасным. Основной язык Microsoft тех лет — C++ — давал полный контроль над памятью через **указатели** (переменные, хранящие адреса в памяти). Это означало:

- программист должен **вручную** освобождать память — забыл → **утечка памяти** (программа постепенно «съедает» всю RAM);
- можно случайно выйти за границы массива и **испортить чужие данные**;
- язык сложен, ошибки проявлялись непредсказуемо, часто во время работы у клиента.

Фактор 3. Интернет требовал новых инструментов. Веб-приложения только зарождались, а старые средства Microsoft (Visual Basic 6, классический ASP) для них не подходили.

Microsoft пыталась адаптировать Java под Windows, но сотрудничества не вышло (Microsoft добавляла Windows-специфичные расширения, Sun увидела в этом нарушение концепции кроссплатформенности — разгорелся судебный конфликт). Microsoft решила **создать свою платформу с нуля**.

1.3. Рождение C# (1998–2002)

- **1998** — внутри Microsoft стартует проект NGWS (Next Generation Windows Services), будущий **.NET**.
- **Июль 2000** — на конференции PDC в Орландо официально представлены язык **C#**, платформа **.NET** и среда исполнения **CLR**.

- **Февраль 2002** — релиз **.NET Framework 1.0** и **C# 1.0** — первую версию языка можно было использовать в реальных проектах.

Главный архитектор языка — Андерс Хейлсберг (датский программист, с 1996 года в Microsoft). До C# он создал:

- **Turbo Pascal** — один из самых популярных компиляторов 1980-х;
- **Delphi** — среду быстрой разработки приложений.

Опыт Хейлсберга объясняет характер C#: язык взял **лаконичный синтаксис у C/C++**, **удобство у Delphi**, а **виртуальную машину и сборку мусора — у Java**, добавив собственные решения.

1.4. Почему язык называется «C#»

- Символ **#** состоит из **четырёх знаков «+»**, наложенных в сетку: намёк на «C++++», то есть развитие языка C++.
- В музыке **C#** («до-диез») — нота на полтона выше «до» (C): символ того, что язык — «шаг вперёд».
- По воспоминаниям участников, рабочее имя проекта было **Cool** (C-like Object Oriented Language — «объектно-ориентированный язык, похожий на C»), но для релиза его сочли неподходящим.

1.5. Стандартизация: язык стал открытым

Чтобы язык приняли за пределами Microsoft, его передали в международные организации стандартизации:

- **Декабрь 2001** — стандарт **ECMA-334** (организация ECMA International);
- **Апрель 2003** — стандарт **ISO/IEC 23270**.

 **Стандарт** — официально утверждённое описание языка: правила, по которым любой желающий может создать свой компилятор C#.

Это позволило появиться независимым реализациям, важнейшая из которых — **Mono** (2004, проект Мигеля де Икасы): кроссплатформенный .NET ещё в те годы, когда официальный .NET работал только под Windows. Позже опыт Mono во многом лёг в основу .NET Core.

1.6. Эволюция языка: таблица версий с объяснениями

Версия	Год	Что добавили	Что это значит простыми словами
1.0	2002	ООП, управляемый код, сборка мусора	Базовый язык: классы, объекты, автоматическая очистка памяти
2.0	2005	Обобщения (generics)	Коллекции «с проверкой типа»: <code>List<int></code> хранит только числа, ошибка ловится при компиляции, а не в работе
3.0	2007	LINQ , лямбда-выражения, <code>var</code>	Запросы к спискам данных прямо в C#: <code>список.Where(x => x.Возраст > 18)</code> — как SQL, но для любых коллекций
4.0	2010	<code>dynamic</code> , именованные/необязательные параметры	Можно вызывать <code>Метод(возраст: 25)</code> — понятно, какой параметр задаётся
5.0	2012	async/await	Программа не «зависает», ожидая долгую операцию (загрузку из интернета, чтение файла)
6.0	2015	Интерполяция строк <code>\$"Привет, {имя}!"</code>	Подстановка переменных в текст без склеивания строк знаком «+»
7.0	2017	Кортежи, pattern matching, <code>out var</code>	Можно вернуть из метода несколько значений; сопоставление данных с образцами
8.0	2019	Nullable reference types	Компилятор предупреждает: «эта переменная может оказаться null» — защита от ошибки «ссылка не указывает на объект»
9.0	2020	records , top-level statements	Короткая запись классов-«контейнеров данных»; программа без обязательного класса <code>Program</code>
10	2021	Global usings, record struct	Общие подключения библиотек для всего проекта
11	2022	Raw string literals, required	Многострочные строки без экранирования; обязательные свойства

Версия	Год	Что добавили	Что это значит простыми словами
12	2023	Primary constructors	Аргументы конструктора прямо в объявлении класса
13	2024	params -коллекции и др.	Улучшения производительности и удобства

⚠ Важно: начиная с C# 6 (2014) компилятор написан заново на C# — он называется **Roslyn**, имеет открытый исходный код и доступен как библиотека (это позволяет инструментам анализировать код: подсказки в Visual Studio, автоформатирование, анализаторы ошибок).

1.7. Каким C# стал сегодня: точные характеристики

- **Мультипарадигменный** — поддерживает несколько стилей написания программ одновременно:
 - *объектно-ориентированный* (программа из объектов — лекция 4);
 - *функциональный* (обработка данных через функции: LINQ, лямбды);
 - *императивный/процедурный* (последовательные инструкции).
- **Строго типизированный** — у каждой переменной есть тип (`int` , `string` ...), и компилятор **не пропустит** операцию над несовместимыми типами. Большинство ошибок ловится **до запуска** программы.
- **Управляемый** — за памятью следит **сборщик мусора (Garbage Collector, GC)**: специальный компонент автоматически находит объекты, которыми программа больше не пользуется, и освобождает их память. Программист в типичном коде не освобождает память вручную.
- **Кроссплатформенный** — программы на C# запускаются на Windows, Linux, macOS, в облаке, на мобильных устройствах и даже в браузере (Blazor/WebAssembly).

// Пример: как выглядит современный C#

```
var names = new[] { "Анна", "Борис", "Виктор" }; // массив строк
```

```
var result = names
```

```
    .Where(n => n.StartsWith("А")) // отфильтровать: оставить начинающиеся с «А»
```

```
    .Select(n => $"Привет, {n}!") // преобразовать каждый элемент
```

```
    .First(); // взять первый результат
```

```
Console.WriteLine(result); // Привет, Анна!
```

1.8. Выводы

1. C# создан в 2000 г. компанией Microsoft под руководством **Андерса Хейлсберга** как ответ на Java и развитие идей C++/Delphi.
2. C# — **язык**, .NET — **платформа**; это разные вещи.
3. Язык стандартизирован (ECMA-334, ISO/IEC 23270), что сделало возможными независимые реализации (Mono) и открытость.
4. C# развивается ежегодно: от «Java-подобного языка для Windows» до универсального современного языка для веба, десктопа, мобильных устройств, игр и облака.

Контрольные вопросы:

1. Чем язык программирования отличается от платформы? Что такое C#, а что такое .NET?
2. Назовите три причины, по которым Microsoft начал разработку нового языка.
3. Какие языки разрабатывал Андерс Хейлсберг до C#?
4. Что означает символ «#» в названии?
5. Зачем нужен сборщик мусора и от какой проблемы C++ он избавляет?

Лекция 2. Платформа .NET: .NET Standard, .NET Core / .NET 5–10, ASP.NET

Словарик к лекции

Термин	Что означает
Runtime (среда исполнения)	Программа, которая выполняет скомпилированный код и предоставляет ему услуги (память, потоки, библиотеки)
API	Набор готовых функций/классов, которыми может пользоваться программист
Backward compatibility (обратная совместимость)	Свойство системы работать со старым кодом: программа, написанная 10 лет назад, запускается на новой версии
Legacy-код	Старый работающий код, который поддерживают, но писать который так уже не будут

Термин	Что означает
EOL (End of Life)	Дата окончания поддержки версии: обновлений безопасности больше не будет

2.1. Проблема: к 2015 году .NET раздробился на несколько платформ

Чтобы понять, зачем нужен .NET Standard, нужно понять, **что случилось**. К 2015 году параллельно существовало несколько несовместимых платформ .NET:

Платформа	Где работала	Для чего создавалась
.NET Framework	только Windows	Классические приложения и веб (заморожена на версии 4.8)
.NET Core	Windows, Linux, macOS	Новая открытая платформа для современных и облачных задач
Mono	Windows, Linux, macOS, мобильные	Независимая кроссплатформенная реализация, основа Xamarin (мобильная разработка)
UWP	только Windows 10	Приложения из Microsoft Store

В чём именно проблема? Вы написали библиотеку для .NET Framework. Смогут ли её использовать разработчики .NET Core? **Неизвестно** — зависит от того, какие функции вы применили. Приходилось проверять вручную или выпускать несколько версий библиотеки. Это тормозило всё сообщество.

2.2. .NET Standard — договор о совместимости

 **Точное определение:** .NET Standard — это **формальная спецификация** (документ-соглашение), которая определяет **единый набор API**. Каждая платформа .NET, заявляющая поддержку некоторой версии .NET Standard, **обязуется** предоставить все API этого набора.

 **Простыми словами:** .NET Standard — это не программа и не платформа, на которой что-то запускается. Это **список обещаний**: «если платформа поддерживает .NET

Standard 2.0, значит, в ней гарантированно есть вот эти классы и методы».

💡 **Аналогия — стандарт розетки:** стандарт евророзетки гарантирует: любые приборы, сделанные под этот стандарт, воткнутся в любую евророзетку в любой стране. Прибор = ваша библиотека, розетки = платформы (.NET Framework, .NET Core, Mono), а сам стандарт розетки — это .NET Standard. Стандарт — **не розетка и не прибор**, это **правило**, которому обязаны соответствовать и те и другие.

Как это выглядело на практике. Автор библиотеки указывал целевой стандарт в файле проекта:

```
<TargetFramework>netstandard2.0</TargetFramework>
```

После этого библиотека гарантированно работала на **любой** платформе, поддерживающей .NET Standard 2.0.

Уровни .NET Standard — важное правило:

Версия	Характеристика
1.0–1.6	Маленький набор API, много уровней — сложно ориентироваться (историческая неудача, быстро исправленная)
2.0	Золотая середина: большой набор API + поддержка почти всеми платформами. Большинство библиотек до сих пор нацелены на неё
2.1	Финальная, самый полный набор; не поддерживается старым .NET Framework

Правило, которое стоит запомнить:

Чем **ниже** версия стандарта → тем **больше** платформ её поддерживают, но тем **меньше** доступно API. Чем **выше** версия → тем **больше** API, но тем **меньше** платформ совместимо.

Судьба .NET Standard. С выходом единой платформы .NET 5 потребность в новых версиях стандарта отпала: теперь совместимость обеспечивает сама платформа (целевая платформа пишется как net8.0 и т. п.). Стандартная библиотека **.NET Standard 2.0** остаётся рабочей — на неё по-прежнему нацелены библиотеки, которым нужна максимальная совместимость со старым .NET Framework.

2.3. .NET Core → .NET 5–10: единая платформа

2.3.1. Зачем создали .NET Core

К 2014 году стало ясно: старый .NET Framework нельзя «починить на ходу» — он глубоко встроен в Windows и обратно совместим с 15-летним кодом. Microsoft приняла решение **переписать платформу с нуля** с четырьмя целями:

- 1. **Кроссплатформенность** — работа на Windows, Linux и macOS (Linux критичен: большинство серверов и облаков работают на нём).
- 2. **Открытый исходный код** — разработка на GitHub, участие сообщества.
- 3. **Производительность** — скорость, критичная для веб-серверов и облака.
- 4. **Модульность** — приложение несёт свои зависимости с собой, ничего не устанавливается «в систему» глобально.

Результат: **.NET Core 1.0** (июнь 2016) → **.NET Core 3.1** (декабрь 2019, последняя версия с этим именем).

2.3.2. Почему после 3.1 вышло .NET 5, а не 4

Причины перехода на «5»:

- пропустить версию «4», чтобы не путать новую платформу со старым **.NET Framework 4.x** (он существует и поддерживается до сих пор как часть Windows);
- показать, что это **главная** версия, объединяющая .NET Core, .NET Framework, Mono/Xamarin в **одну платформу**.

С версии 5 нумерация единая: **.NET 6, 7, 8, 9, 10...** — новая версия выходит каждый год в ноябре.

2.3.3. Версии и их поддержка

Версия	Год выхода	Тип	Статус поддержки*
.NET 5	2020	STS	поддержка закончилась (май 2022)
.NET 6	2021	LTS	поддержка закончилась (ноябрь 2024)
.NET 7	2022	STS	поддержка закончилась (май 2024)
.NET 8	2023	LTS	поддерживается до ноября 2026
.NET 9	2024	STS	поддерживается до мая 2026

Версия	Год выхода	Тип	Статус поддержки*
.NET 10	2025	LTS	долгосрочная поддержка

 **STS** (Standard Term Support) — стандартный срок поддержки: **18 месяцев**.  **LTS** (Long Term Support) — долгосрочная поддержка: **3 года**. Обновления безопасности и исправления выпускаются весь срок.

* Даты приводятся на момент составления курса — актуальные данные всегда проверяйте на официальном сайте Microsoft (dotnet.microsoft.com).

 **Правило для практики:** для учебных проектов берите последнюю версию; для «боевых» корпоративных проектов — последнюю **LTS**.

2.3.4. Ключевые свойства .NET 5+

Свойство	Что это значит для вас
Кроссплатформенность	Один и тот же код работает на Windows, Linux, macOS; публикуется в облако, запускается в Docker
Открытый исходный код	Весь код платформы доступен на GitHub (github.com/dotnet/runtime) — можно читать, предлагать изменения
Высокая производительность	Платформа регулярно занимает первые места в независимых замерах скорости веб-серверов (TechEmpower)
Единый SDK	Один комплект инструментов для всех типов приложений: сайт, API, десктоп, мобильное, игра
AOT-компиляция (Native AOT)	Возможность скомпилировать программу сразу в машинный код — быстрый старт, не требуется установка рантайма
Гибкое развёртывание	Можно распространять: (а) требуя установленный .NET, или (б) упаковывая всё нужное в один исполняемый файл

2.3.5. Что позволяет создавать .NET 5+



ASP.NET	Десктоп	Мобильные	Игры	ИИ/IoT
Core:	(WPF,	(MAUI –	(Unity –	(ML.NET –
сайты,	WinUI,	Android/	игровые	машинное
API,	консоль)	iOS из одного	движки)	обучение)
real-time		кода)		

2.3.6. Плюсы и минусы .NET 5+

✅ Плюсы	❌ Минусы
Кроссплатформенность без ограничений	Новый релиз каждый год — нужно следить за обновлениями
Высокая скорость (топ бенчмарков)	Старый код .NET Framework нельзя просто перенести — только переписать/адаптировать
Огромная стандартная библиотека + каталог пакетов NuGet	Сама платформа «тяжёлая» для крошечных утилит (частично решается Native AOT)
Предсказуемый годовой цикл релизов, понятные LTS	
Отличные бесплатные инструменты: Visual Studio Community, VS Code, Rider (для студентов бесплатно)	
Высокий спрос на рынке труда	

2.4. ASP.NET Core — веб-разработка на .NET

2.4.1. Минимум теории: как работает веб

Прежде чем говорить о фреймворке, разберёмся с механикой:

1. **Браузер** (клиент) отправляет **HTTP-запрос**: «дай мне страницу /about» или «сохрани форму заказа».
2. **Веб-сервер** (программа на удалённом компьютере) принимает запрос, обрабатывает его (например, лезет в базу данных).
3. Сервер возвращает **HTTP-ответ**: HTML-страницу, данные в формате JSON, картинку и т. д.

 **Веб-фреймворк** — готовый набор инструментов, который избавляет от ручной работы с низкоуровневыми деталями HTTP: он сам разбирает запросы, маршрутизирует их («какой код обработать этот адрес»), формирует ответы.

2.4.2. История ASP.NET

Этап	Год	Что было
ASP.NET WebForms	2002	Разработка «как в настольных программах»: рисуешь кнопки мышкой, пишешь обработчики нажатий. Скрытое поле ViewState хранило состояние страницы — страницы «покалечили» вес, а сгенерированный HTML был почти неуправляемым. Работало только на Windows/IIS
ASP.NET MVC	2009	Переход на паттерн Model-View-Controller : полный контроль над HTML, разделение логики и представления, удобное тестирование
ASP.NET Core	2016	Полная переработка : кроссплатформенность, открытый код, высокая скорость. С тех пор развивается вместе с .NET 5+

2.4.3. Из чего состоит ASP.NET Core сегодня

Компонент	Задача	Пример использования
MVC	Веб-сайты по паттерну Model-View-Controller	Корпоративные порталы, интернет-магазины
Razor Pages	Упрощённая постраничная модель	Небольшие сайты, формы, лендинги
Web API	Сервисы, отдающие данные (обычно JSON)	Серверная часть для мобильных приложений и сайтов на React/Angular/Vue
Blazor	Интерактивный интерфейс на C# вместо JavaScript	Админ-панели, корпоративные приложения
SignalR	Связь в реальном времени (сервер сам «толкает» данные клиенту)	Чаты, уведомления, биржевые графики

Компонент	Задача	Пример использования
gRPC	Быстрый обмен сообщениями между сервисами	Связь микросервисов внутри компании
Minimal API	Пишем API буквально несколькими строками	Микросервисы, прототипы

```
// Пример: полноценный веб-сервер на Minimal API — 5 строк
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Привет, мир!"); // страница по адресу "/"
app.MapGet("/users/{id:int}", (int id) => $"Пользователь №{id}"); // с параметром

app.Run(); // сервер запущен и ждёт запросов
```

 **Что здесь произошло:** мы объявили **маршруты** — правила «при запросе по такому-то адресу выполни вот эту функцию». `MapGet` означает «реагировать на HTTP GET-запрос» (именно GET браузер посылает, когда открывает страницу).

2.4.4. Плюсы и минусы ASP.NET Core

 Плюсы	 Минусы
Одна из самых быстрых веб-платформ в мире	Порог входа выше, чем у PHP/Python: строгая типизация, компиляция, больше «правил»
Многое встроено «из коробки»: DI-контейнер, конфигурация, логирование, кэширование, аутентификация	Готовых шаблонов сайтов/CMS меньше, чем в WordPress или Laravel
Работает на Linux, в Docker, Kubernetes, любых облаках (Azure, AWS, Яндекс Облако)	
Конвейер middleware — запрос проходит через цепочку обработчиков, которые легко настраивать	
Первоклассная документация (learn.microsoft.com)	

2.5. Где всё это работает: платформы и хостинг

Среда	Как запускается
Windows	Встроенный сервер Kestrel (+ опционально IIS как «реверс-прокси»)
Linux	Kestrel + nginx/Apache перед ним, управление через systemd
macOS	Kestrel (в основном для разработки)
Docker / Kubernetes	Официальные образы <code>mcr.microsoft.com/dotnet</code>
Облачные платформы	Azure, AWS, Google Cloud, Yandex Cloud

 **Kestrel** — встроенный кроссплатформенный веб-сервер ASP.NET Core: программа сама умеет принимать HTTP-запросы, отдельный веб-сервер не обязателен.

2.6. Выводы

1. **.NET Standard** — это не платформа, а **спецификация-договор** о наборе API; она решила проблему несовместимости нескольких платформ .NET. Сегодня её роль выполняет единый .NET (`net8.0` и т. д.).
2. **.NET Core** → **.NET 5+** — единая кроссплатформенная открытая платформа: релиз каждый ноябрь, версии делятся на LTS (3 года) и STS (18 месяцев).
3. **ASP.NET Core** — веб-фреймворк этой платформы: сайты, API, real-time, микросервисы; в топе мировых замеров производительности.
4. **Практическое правило:** новые проекты — на .NET LTS (сейчас .NET 8 или новее); .NET Framework 4.8 — только для поддержки существующего старого кода.

Контрольные вопросы:

1. Чем .NET Standard отличается от .NET Core? Почему стандарт — это не платформа?
2. Сформулируйте правило «чем ниже версия стандарта...» до конца.
3. Назовите минимум три причины появления .NET Core.
4. Почему после .NET Core 3.1 вышло .NET 5, а не 4?
5. Что произойдёт при запросе браузера к адресу `/users/7` в примере Minimal API? Какой метод и что вернёт?

6. Чем Web API отличается от Blazor по назначению?

Лекция 3. Бинарные данные, системы счисления и то, как компьютер «понимает» программы

Словарик к лекции

Термин	Что означает
Данные	Любая информация, представленная в виде, пригодном для хранения и обработки
Память	Место, где хранятся данные и работающая программа; состоит из огромного числа пронумерованных ячеек по 1 байту
Машинный код	Последовательность чисел-инструкций, которые процессор умеет выполнять напрямую
Компиляция	Перевод исходного кода в другой вид (машинный или промежуточный) до запуска программы
Интерпретация	Выполнение исходного кода по ходу , инструкция за инструкцией

3.1. Фундамент: компьютер — машина с двумя состояниями

Внутри процессора и памяти миллиарды **транзисторов** — электронных переключателей, каждый из которых может быть в одном из двух состояний: **ток есть / тока нет**.

 **Бит (bit, binary digit)** — минимальная единица информации; принимает одно из двух значений: **0** («нет сигнала») или **1** («есть сигнал»).

 **Аналогия:** бит — это лампочка. Она либо выключена (0), либо включена (1). Третьего не бывает.

Из битов собирается всё: числа, текст, музыка, видео. Иерархия единиц:

Единица	Состав	Пример
1 бит	0 или 1	одна лампочка
1 байт	8 бит	минимальная ячейка, по которой процессор адресует память
1 Кбайт	1024 байта	почему 1024, а не 1000? Потому что 2^{10} — «круглое» число в двоичной системе
1 Мбайт	1024 Кбайт	
1 Гбайт	1024 Мбайт	

Почему в C# тип `byte` хранит значения от 0 до 255? Байт — это 8 битов. Восемь единиц (`11111111`) — максимальное число, которое в них помещается:

$$11111111_2 = 128+64+32+16+8+4+2+1 = 255$$

Всего различных значений: $2^8 = 256$ (от 0 до 255)

3.2. Системы счисления

 **Система счисления** — способ записи чисел с помощью ограниченного набора символов (цифр). **Основание** системы = количество различных цифр в ней.

Система	Основание	Цифры	Где встречается
Двоичная	2	0, 1	«родной язык» железа
Восьмеричная	8	0–7	права доступа в Linux
Десятичная	10	0–9	привычная людям запись
Шестнадцатеричная	16	0–9, A, B, C, D, E, F	цвета, адреса памяти, содержимое файлов

 **Важно понять:** система счисления — это только **способ записи** числа, а не само число. Число «пятнадцать» одно, но записывается как `15` (в десятичной), `1111` (в

двоичной), F (в шестнадцатеричной). Это как «15», «пятнадцать» и «XV» — одна и та же величина.

3.2.1. Почему программисты любят шестнадцатеричную систему

Ключевое соотношение: **одна hex-цифра кодирует ровно 4 бита** (потому что $16 = 2^4$). Значит, **байт всегда записывается ровно двумя hex-цифрами**:

```
Байт:      1111 0000
           ↓   ↓
Hex:       F   0      →  $F0_{16} = 240_{10}$ 
```

Hex — это удобная «сокращённая запись» двоичных чисел: без длинных строк из единиц и нулей, но с прямым пересчётом в биты.

Пример из практики — цвет в веб-дизайне: #FF6A00

- FF = 255 → красная составляющая (максимум);
- 6A = 106 → зелёная;
- 00 = 0 → синяя (нет). Итог: насыщенный оранжевый.

3.2.2. Перевод чисел: два алгоритма, которые нужно уметь

Десятичная → двоичная (делим на 2, собираем остатки снизу вверх):

```
13 ÷ 2 = 6,  остаток 1  ↑
 6 ÷ 2 = 3,  остаток 0  ↑  читаем
 3 ÷ 2 = 1,  остаток 1  ↑  снизу
 1 ÷ 2 = 0,  остаток 1  ↑  вверх
Ответ:  $13_{10} = 1101_2$ 
```

Двоичная → десятичная (каждый бит — вклад по степени двойки):

Позиции битов (справа налево): $2^3=8$, $2^2=4$, $2^1=2$, $2^0=1$

$1101_2 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13_{10}$

3.2.3. Запись чисел в разных системах в C#

```
int dec = 255;           // обычная запись
int hex = 0xFF;          // префикс 0x — шестнадцатеричная
```

```
int bin = 0b1111_1111; // префикс 0b – двоичная; "_" – разделитель для читаемости

Console.WriteLine(dec == hex && hex == bin); // True – это ТРИ ЗАПИСИ ОДНОГО ЧИСЛА

// Обратные преобразования – числа в строку:
Console.WriteLine(Convert.ToString(255, 2)); // "11111111" (в двоичную)
Console.WriteLine(255.ToString("X")); // "FF" (в шестнадцатеричную)
```

3.3. Как компьютер хранит целые числа

3.3.1. Целочисленные типы C#

Тип	Размер	Диапазон	Когда использовать
byte	1 байт	0 ... 255	байты файла, «сырые» данные
sbyte	1 байт	−128 ... 127	редко
short / ushort	2 байта	−32 768 ... 32 767	экономия памяти в больших массивах
int / uint	4 байта	≈ ±2,1 млрд	выбор по умолчанию для целых
long / ulong	8 байт	≈ ±9,2 квинтиллиона	большие счётчики, ID, время в миллисекундах

3.3.2. Отрицательные числа: дополнительный код

Вопрос: где в байте «минус»? Ответ: **старший (левый) бит** работает как знак: 0 — плюс, 1 — минус. Но хранится отрицательное число не «со знаком минус», а в **дополнительном коде**:

 **Дополнительный код (two's complement)** — стандартный способ представления отрицательных чисел: чтобы получить $-X$, нужно записать X , инвертировать все биты ($0 \leftrightarrow 1$) и прибавить 1.

Получим -5 в 8 битах:

Шаг 1. Число 5: 00000101

Шаг 2. Инвертируем биты: 11111010

Шаг 3. Прибавляем 1: 11111011 ← это -5

Зачем такая сложность? С дополнительным кодом процессору достаточно **одной** операции — сложения — и для «+», и для «-». Проверим:

```
7 + (-5):
  00000111  (7)
+ 11111011  (-5)
= 100000010 ← девятый бит не помещается и отбрасывается
  00000010   = 2 ✓ Верно!
```

💡 **Аналогия — одометр (счётчик пробега) автомобиля:** он катится по кругу: 999 998 → 999 999 → 000 000. «Откат назад» через ноль — то же самое, что вычитание. Дополнительный код устроен по этому принципу: у чисел есть «потолок», и минус живёт сразу «под потолком».

3.3.3. Переполнение: что будет, если выйти за границу

```
int max = int.MaxValue;      // 2 147 483 647 — максимум для int
Console.WriteLine(max + 1);  // -2147483648 ← НЕ ошибка, а «перекат через одометр»!
```

По умолчанию C# **молча** перекатывает значение — это источник трудноуловимых багов. Защита — оператор `checked` :

```
checked
{
    int x = int.MaxValue + 1; // здесь будет выброшено OverflowException
}
```

3.4. Дробные числа: почему $0.1 + 0.2 \neq 0.3$

3.4.1. Формат IEEE 754

📖 `float` и `double` хранят числа в формате **IEEE 754**: число записывается в виде **знак × мантисса × $2^{\text{экспонента}}$** — это «научная запись» (как $1,496 \times 10^8$ км — расстояние до Солнца), но в двоичном виде.

Тип	Размер	Примерная точность	Назначение
float	4 байта	~6–9 значащих цифр	графика, игры (быстрее, менее точен)
double	8 байт	~15–17 значащих цифр	научные расчёты, физика — выбор по умолчанию
decimal	16 байт	28–29 цифр, десятичное основание	деньги и всё, где нужна точная десятичная запись

Откуда берётся знаменитая погрешность: в двоичной системе дробь 1/10 — **бесконечная** периодическая дробь (как 1/3 = 0,3333... в десятичной). Приходится «обрезать» — отсюда хвост:

```
Console.WriteLine(0.1 + 0.2);    // 0.30000000000000004  (double – погрешность)
Console.WriteLine(0.1m + 0.2m); // 0.3                (decimal – точен)
```

 **Правило на всю карьеру:** денежные суммы считайте *только* в `decimal`. `double` для денег недопустим — копейки будут «теряться» и «появляться».

3.5. Как компьютер хранит текст: кодировки

Компьютер умеет хранить только числа. Значит, тексту нужен **соглашённый** перевод «символ → число». Такое соглашение называется **кодировкой**:

 **Кодировка** — таблица соответствий между символами и числовыми кодами.

Кодировка	Устройство	История
ASCII (1963)	7 бит → 128 символов: латиница, цифры, знаки	Хватало только англоязычным
Windows-1251 , KOI-8 и др.	1 байт на символ; первые 128 кодов — как ASCII, старшие — «национальные» буквы	Проблема: в разных странах старшие коды означали РАЗНЫЕ буквы — возникали «кракозябры»
Unicode	Единая мировая таблица: 1 114 112 кодовых позиций для всех языков, эмодзи, символов	Решил проблему «у каждого своя таблица»

Кодировка	Устройство	История
UTF-8	Способ записи Unicode переменной длины : латиница — 1 байт, кириллица — 2 байта, эмодзи — 4 байта	Стандарт интернета: экономен для английского текста и универсален

```
char c = 'A';
Console.WriteLine((int)c);      // 65 — так 'A' выглядит «для компьютера»
Console.WriteLine((char)1040); // A — обратное преобразование кода в символ

string text = "Привет";
byte[] bytes = System.Text.Encoding.UTF8.GetBytes(text); // строка → байты
Console.WriteLine(string.Join(" ", bytes));
// 208 159 209 128 208 184 208 178 208 181 209 130
// 6 букв кириллицы = 12 байт: каждая буква занимает ДВА байта (208/209 — маркер пары)
```

⚠ Практический вывод: *string в памяти — это на самом деле набор чисел; «длина строки в символах» и «размер в байтах» — **разные вещи**. Это важно при чтении файлов, передаче по сети, хранении в базе данных.*

3.6. Битовые операции — работа с битами напрямую

Иногда нужно управлять отдельными битами (это быстро и экономно):

```
int a = 0b1100; // 12
int b = 0b1010; // 10

Console.WriteLine(a & b); // 8 (0b1000) — И:      бит = 1, только если 1 у ОБОИХ чисел
Console.WriteLine(a | b); // 14 (0b1110) — ИЛИ:    бит = 1, если есть хотя бы одна 1
Console.WriteLine(a ^ b); // 6 (0b0110) — XOR:     бит = 1, если биты РАЗЛИЧНЫ
Console.WriteLine(~a);    // -13 — НЕ:           все биты инвертированы
Console.WriteLine(a << 1); // 24 (0b11000) — сдвиг влево на 1 = умножение на 2
Console.WriteLine(a >> 2); // 3 (0b0011) — сдвиг вправо на 2 = деление на 4
```

Классическое применение — упаковка прав в одно число (битовые флаги):

```
[Flags]
enum Permissions
```

```

{
    None    = 0,        // 000
    Read    = 1,        // 001
    Write   = 2,        // 010
    Delete  = 4         // 100  (каждому праву — свой бит!)
}

var user = Permissions.Read | Permissions.Write; // 011 — чтение + запись
bool canDelete = user.HasFlag(Permissions.Delete); // False

```

Вместо трёх полей `bool` храним одно число — так работают системы прав в ОС и базах данных.

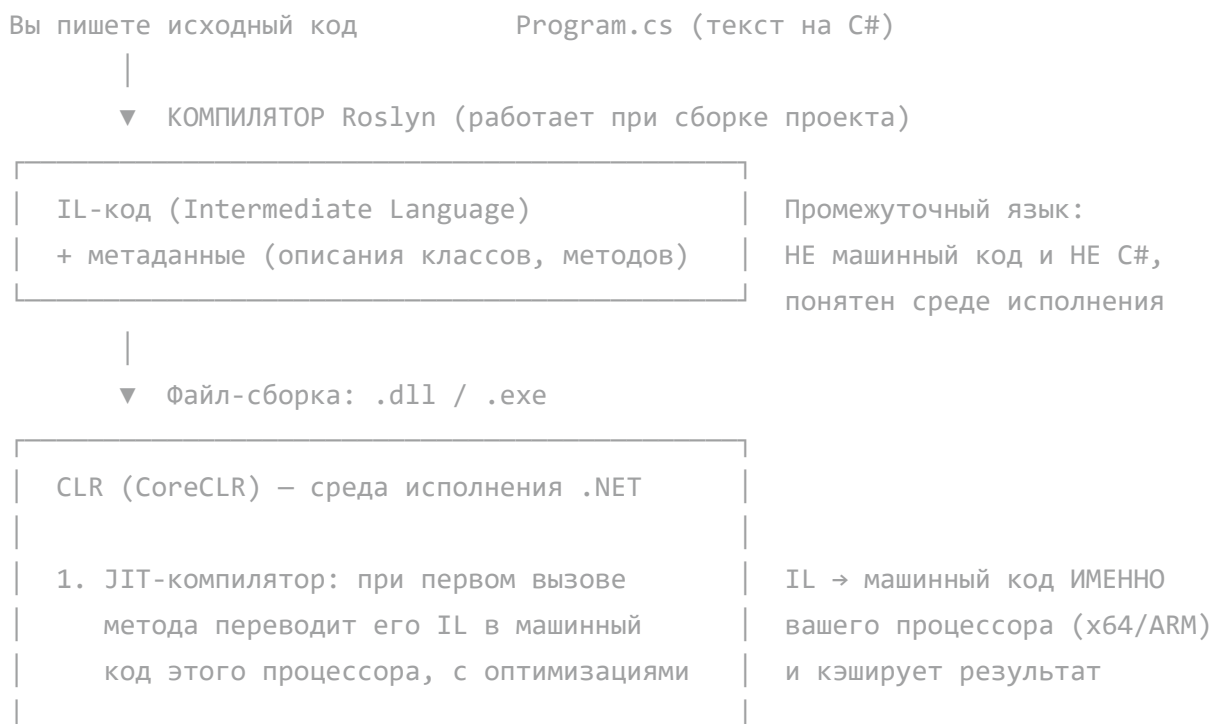
3.7. Главное: как компьютер исполняет программу

3.7.1. Проблема

Процессор понимает только **машинный код** — числа, кодирующие элементарные команды («сложить», «записать в память», «перейти»). Причём набор команд **свой у каждого семейства процессоров**: x64 (типичные ПК), ARM (телефоны, новые Mac). Написанная для x64 программа на ARM «с самого начала» не заработает.

Человек не пишет в машинном коде. Значит, между текстом на C# и процессором нужен **переводчик**. У .NET этот перевод двухступенчатый.

3.7.2. Схема выполнения программы на C#



2. Garbage Collector: освобождает память от ненужных объектов
3. Проверка типобезопасности



ПРОЦЕССОР выполняет машинный код: 01001101...

Разбор ключевых понятий:

Термин	Точное значение
IL (Intermediate Language)	Промежуточный язык .NET. Компиляторы C#, F#, VB.NET переводят код в общий IL — поэтому все эти языки работают на одной платформе
Сборка (assembly)	Файл .dll / .exe , содержащий IL-код и метаданные. Основная единица распространения кода
CLR (Common Language Runtime)	Среда исполнения .NET: выполняет IL через JIT, управляет памятью и типами
JIT (Just-In-Time)	Компиляция IL в машинный код во время работы программы , по мере вызова методов; результат кэшируется — повторный вызов уже не компилируется
AOT (Ahead-Of-Time)	Полная компиляция в машинный код до запуска (режим Native AOT): мгновенный старт, платформа не нужна на целевой машине
GC (Garbage Collector)	Сборщик мусора: автоматически находит недостижимые объекты и освобождает их память

💡 **Аналогия — перевод книги.** Публикация книги на международной конференции:

- **AOT** — книга заранее переведена целиком на нужный язык (быстро читать, но нужен отдельный перевод под каждый язык);
- **JIT** — переводчик-профессионал переводит по ходу, но **записывает** каждый переведённый фрагмент: при повторном чтении того же места перевод выдаётся мгновенно, к тому же он учитывает реакцию зала (оптимизирует под конкретную аудиторию). Так работает .NET;

- **Интерпретация** — синхронист переводит устно по фразе и ничего не записывает: медленно при повторении. Так работает классический Python.

3.7.3. Один метод на трёх уровнях

```
// Уровень 1. C# — то, что пишет человек
int Square(int x) => x * x;
```

```
// Уровень 2. IL — то, что хранится в .dll (упрощённо; смотреть: утилита ildasm)
ldarg.1      // положить аргумент x на стек
ldarg.1      // положить x ещё раз
mul          // перемножить два верхних значения
ret          // вернуть результат
```

```
; Уровень 3. Машинный код x64 — то, что исполняет процессор (упрощённо)
mov eax, ecx    ; скопировать x в регистр eax
imul eax, eax   ; eax = eax * eax
ret             ; выход
```

Почему так? IL один для всех платформ, а машинный код JIT генерирует **под конкретный процессор** — поэтому сборка .dll с Windows-компьютера без перекомпиляции работает на Linux/ARM: её «переводит» местная среда исполнения.

3.8. Что такое «скрипты»?

 **Скрипт** — программа, которая исполняется **интерпретатором по ходу чтения кода**, без отдельного этапа компиляции в исполняемый файл. Файл скрипта одновременно является и «исходником», и «программой».

Сравнение трёх подходов исполнения:

Подход	Механизм	Примеры	Когда удобен
Компиляция (AOT)	Код → машинный код заранее	C, C++, Rust, Go, Native AOT	Максимальная скорость, не нужен рантайм

Подход	Механизм	Примеры	Когда удобен
VM + JIT	Код → промежуточный код → машинный код при запуске	C# , Java, Kotlin	Баланс скорости и кроссплатформенности
Интерпретация	Код выполняется построчно	Python, JavaScript, PHP, Bash	Быстрое написание, автоматизация, склейка систем

Типичные примеры скриптов: автоматизация рутинных задач (`.bat` , Bash), сборка и развёртывание, обработка файлов, мелкие утилиты.

C# умеет работать в «скриптовом» стиле — для быстрых экспериментов:

```
// файл script.csx (C# Script): НЕ требуется класс и метод Main
Console.WriteLine("Я — скрипт на C#!");
```

```
# запуск через сторонний инструмент dotnet-script:
dotnet script script.csx
```

А начиная с C# 9 (2020) обычная консольная программа может выглядеть как скрипт:

```
// Program.cs — компилируется нормально, но без «обвязки» из класса Program и static void
Console.WriteLine("Hello, .NET!");
```

 **Для точности:** под капотом компилятор всё равно сгенерирует класс с методом `Main` — *top-level statements* лишь избавляют вас от ручного написания обвязки. Это настоящая компилируемая программа, а не интерпретируемый скрипт.

3.9. Выводы

1. Всё в компьютере — числа, текст, звук, видео — это **биты и байты**; понятия «двоичная система» и «байт» — фундамент профессии.
2. **Шестнадцатеричная система** — компактная запись байтов: 1 hex-цифра = 4 бита.
3. Отрицательные числа хранятся в **дополнительном коде** — так вычитание сводится к сложению.

4. `double` хранит дроби приближённо (двоичные дроби часто бесконечны) — **деньги считаем в `decimal`**.
5. Цепочка исполнения C#: **исходный код** → **Roslyn** → **IL (.dll)** → **CLR/JIT** → **машинный код процессора**.
6. **Скрипт** — программа, исполняемая интерпретатором без компиляции в отдельный файл; C# поддерживает и скриптовый стиль (`.csx` , `top-level statements`).

Практические задания:

1. Переведите вручную: $45_{10} \rightarrow ?_2$; $110110_2 \rightarrow ?_{10}$; $0x2F \rightarrow ?_{10}$; $FF_{16} \rightarrow ?_2$.
2. Получите представление числа **-12** в дополнительном коде (8 бит) и проверьте сложением $20 + (-12) = 8$.
3. Объясните, почему $0.1 + 0.2 \neq 0.3$ для `double` , и покажите, что для `decimal` равенство выполняется.
4. Сколько байт займёт ваша фамилия в UTF-8? Проверьте кодом через `Encoding.UTF8.GetBytes` .
5. Напишите метод, который проверяет, включён ли n-й бит в числе (подсказка: $(x \gg n) \& 1$).

Лекция 4. Основы ООП в C#: объекты и четыре принципа объектно-ориентированного программирования

Словарик к лекции

Термин	Что означает
Класс	Описание типа данных: из чего состоит объект и что умеет делать
Объект (экземпляр)	Конкретный «представитель» класса, созданный в памяти
Член класса	Любой элемент внутри класса: поле, свойство, метод, конструктор

Термин	Что означает
Инвариант	Условие, которое всегда должно быть истинно для корректного объекта (например: возраст ≥ 0)

4.1. Зачем придумали ООП

До ООП господствовал **процедурный подход**: программа — это данные (переменные) и функции, которые эти данные обрабатывают, причём данные и функции **разделены**. На маленьких программах это работает. Но когда проект растёт:

- любая функция может изменить любые данные → «кто и когда испортил переменную» — неразрешимый вопрос;
- понятия реального мира («пользователь», «заказ») нигде не описаны как единое целое — они размазаны по сотням переменных и функций;
- код трудно переиспользовать.

 **ООП (объектно-ориентированное программирование)** — парадигма, в которой программа строится из **объектов**, каждый из которых объединяет **данные** (состояние) и **методы** (поведение) и отвечает за собственную корректность.

 **Ключевая идея:** в реальном мире мы всегда думаем объектами: «пользователь Анна, у неё есть имя и баланс, она может пополнить счёт». ООП позволяет писать код, который устроен так же.

4.2. Класс и объект: чертёж и дом

 **Класс** — описание типа: какие данные хранит объект и какие действия выполняет. **Объект (экземпляр класса)** — конкретный представитель этого типа, созданный в памяти оператором `new`.

 **Аналогия:** класс — **чертёж** дома или **формочка** для печенья. Объект — построенный дом, испечённое печенье. Чертёж один — домов много; изменение чертежа не влияет на уже построенные дома, а каждый дом живёт своей жизнью.

```
class Car                                // КЛАСС — чертёж
{
    public string Model;                 // данные объекта
    public int Speed;

    public void Accelerate() // поведение объекта
    {
        Speed += 10;
    }
}

// СОЗДАЁМ ОБЪЕКТЫ оператором new — каждый в своей области памяти:
Car car1 = new Car { Model = "Tesla", Speed = 0 };
Car car2 = new Car { Model = "Lada", Speed = 60 };

car1.Accelerate();                      // разгоняем ТОЛЬКО car1

Console.WriteLine(car1.Speed);           // 10
Console.WriteLine(car2.Speed);           // 60 — car2 не тронули: объекты независимы
```

4.3. Из чего состоит объект: поля, свойства, методы

4.3.1. Поля (fields) — внутреннее состояние

 **Поле** — переменная, объявленная внутри класса; хранит часть состояния объекта.

```
class Person
{
    private string name;    // private — доступно ТОЛЬКО внутри класса
    private int age;

    public static int Population = 0; // static — общее для ВСЕХ объектов класса,
}                                     // хранится в единственном экземпляре
```

Два правила хорошего стиля:

1. Поля делают `private` — снаружи к ним никто не должен лезть напрямую (объяснение — в разделе про инкапсуляцию).
2. Частные поля по соглашению называют с подчёркиванием: `_name`, `_age` — чтобы в коде сразу видеть, что это внутренние данные.

4.3.2. Свойства (properties) — контролируемый доступ к данным

 **Свойство** — член класса, предоставляющий доступ к данным через специальные блоки **get** (чтение) и **set** (запись). Снаружи свойство выглядит как поле (`obj.Name = "..."`), но внутри это методы — и в них можно выполнять **любую логику**.

 **Аналогия:** поле — это сейф в кабинете; свойство — **окно приёма** в этот кабинет. Через окно нельзя влезть в сейф руками — можно только подать заявку, которую сотрудник **проверит** и выполнит по правилам.

```
class Person
{
    private string _name;
    private int _age;

    // 1) ПОЛНОЕ свойство: логика в обоих блоках
    public string Name
    {
        get { return _name; }
        set
        {
            if (string.IsNullOrEmpty(value)) // value — специальное слово:
                throw new ArgumentException("Имя пусто"); // то, что присваивают
            _name = value.Trim();
        }
    }

    // 2) Свойство с проверкой инварианта
    public int Age
    {
        get { return _age; }
        set
        {
            if (value < 0 || value > 150)
                throw new ArgumentException("Некорректный возраст");
            _age = value;
        }
    }

    // 3) ВЫЧИСЛЯЕМОЕ свойство: не хранится, а считается на лету; только чтение
    public bool IsAdult => _age >= 18;
}
```

```
// 4) АВТОСВОЙСТВО: компилятор сам создаёт скрытое поле-хранилище
public string Email { get; set; }

// 5) АВТОСВОЙСТВО с ЗАКРЫТЫМ setter: менять может только сам класс
public string Id { get; private set; }
}
```

Почему свойства, а не публичные поля? Это не каприз — это защита расширяемости:

Публичное поле	Свойство
Значение пишется в память без проверок → инварианты нарушаются	В <code>set</code> — валидация
Нельзя отладить чтение (точка останова не сработает)	Можно ставить точки останова в <code>get</code> / <code>set</code>
Превратить поле в свойство позже = перекомпилировать всё, что его использует	Свойство можно расширять, не ломая внешний код

4.3.3. Методы (methods) — поведение

 **Метод** — функция, объявленная внутри класса; описывает действие, которое объект умеет выполнять. Методы — единственный «законный» способ изменять состояние объекта снаружи.

```
class BankAccount
{
    public decimal Balance { get; private set; } // снаружи – только чтение

    public void Deposit(decimal amount)           // внести деньги
    {
        if (amount <= 0)
            throw new ArgumentException("Сумма должна быть положительной");
        Balance += amount;                        // состояние меняется ТОЛЬКО здесь,
    }                                              // и только по правилам класса

    public bool Withdraw(decimal amount)          // снять деньги
    {
        if (amount > Balance) return false;      // метод может вернуть результат
        Balance -= amount;
        return true;
    }
}
```

}

}

4.3.4. Конструктор — правила «рождения» объекта

 **Конструктор** — специальный метод, имя которого совпадает с именем класса; вызывается автоматически при создании объекта (`new`) и обязан привести объект в корректное начальное состояние.

```
class Person
{
    public string Name { get; }
    public int Age { get; }

    public Person(string name, int age)    // конструктор с параметрами
    {
        Name = name;
        Age  = age;
    }
}

var p = new Person("Анна", 25);  // конструктор вызывается здесь
```

Сводная таблица членов объекта

Член	Что делает	Аналогия
Поле	Хранит данные (внутренне)	Сейф
Свойство	Контролируемый доступ к данным	Окно приёма с проверкой заявок
Метод	Выполняет действие	Кнопка «Сделать X»
Конструктор	Инициализирует объект при создании	Акт регистрации при рождении

4.4. Четыре принципа ООП

Запоминалка: **А-И-Н-П** — **А**бстракция, **И**нкапсуляция, **Н**аследование, **П**олиморфизм.

Принцип 1. АБСТРАКЦИЯ (Abstraction)

 **Точное определение:** абстракция — принцип, при котором для работы с объектом предоставляется набор **существенных для задачи** характеристик и действий, а детали реализации скрываются. Абстракция задаёт **что** объект делает, не раскрывая **как**.

 **Простыми словами:** мы описываем объект с той стороны, с которой он нужен в нашей программе, и отбрасываем всё лишнее. В программе «таксопарк» у машины важны марка, водитель, пробег — но точно не химический состав бензина.

 **Аналогия:** водитель управляет автомобилем через **руль, педали и коробку передач**. Он не обязан знать, как устроен впрыск топлива — интерфейс управления скрывает механику. Более того: устройство бензинового и электрического двигателя разное, а интерфейс (педаль газа) — одинаковый.

Инструменты абстракции в С# — интерфейсы и абстрактные классы:

```
// ИНТЕРФЕЙС — чистая абстракция: только договор «что умеем», без единой строчки реализации
interface IPaymentMethod
{
    bool Pay(decimal amount);    // «умею оплатить сумму; верну успех/неудачу». КАК — не с
}

// Каждая реализация скрывает СВОИ детали:
class CreditCardPayment : IPaymentMethod
{
    public bool Pay(decimal amount)
    {
        // здесь мог бы быть запрос к банковскому шлюзу...
        Console.WriteLine($"Оплата картой: {amount} руб.");
        return true;
    }
}

class CryptoPayment : IPaymentMethod
{
    public bool Pay(decimal amount)
    {
        // здесь — работа с блокчейн-сетью...
```

```
        Console.WriteLine($"Оплата криптовалютой: {amount} руб.");  
        return true;  
    }  
}
```

Зачем: код выше можно писать и читать в терминах предметной области («оплата»), не погружаясь в байты и протоколы. Это снижает сложность — главную врага больших систем.

Принцип 2. ИНКАПСУЛЯЦИЯ (Encapsulation)

 **Точное определение:** инкапсуляция — объединение данных и методов в классе с одновременным **сокрытием внутреннего состояния**: прямого доступа к данным нет, изменение возможно только через публичные методы/свойства, которые контролируют корректность (инварианты).

 **Простыми словами:** объект — «чёрный ящик»: снаружи видны только кнопки (публичные методы и свойства), а внутренности (приватные поля) защищены. Объект **сам** следит, чтобы его данные всегда были корректными.

 **Аналогия — банкомат:** вы не можете залезть в хранилище банка напрямую. Вам доступны кнопки: «снять», «внести», «показать баланс». Банкомат **сам** проверит PIN, достаточность средств и лимиты. Хранилище — приватное поле; кнопки — публичные методы.

```
class BankAccount  
{  
    private decimal _balance;                // СПРЯТАНО: снаружи недоступно  
  
    public decimal Balance => _balance;      // только ЧТЕНИЕ (нет блока set)  
  
    public void Deposit(decimal amount)  
    {  
        if (amount <= 0)  
            throw new ArgumentException("Некорректная сумма");  
        _balance += amount;  
    }  
  
    public void Withdraw(decimal amount)  
    {
```

```
        if (amount > _balance)
            throw new InvalidOperationException("Недостаточно средств");
        _balance -= amount;
    }
}
```

Инкапсуляция на практике — попытка обмана пресекается компилятором:

```
var acc = new BankAccount();
// acc._balance = -1000; // ОШИБКА КОМПИЛЯЦИИ: поле приватное
// acc.Balance = 1_000_000; // ОШИБКА: у свойства нет setter
acc.Deposit(100); // единственный путь — через правила класса
```

Инструмент контроля доступа — модификаторы доступа:

Модификатор	Кому доступен член
public	Всем
private	Только внутри этого класса
protected	Этому классу и его наследникам
internal	Любому коду внутри текущего проекта (сборки)
protected internal	Проекту + наследникам
private protected	Наследникам, но только внутри проекта

Зачем нужна инкапсуляция (три причины):

- 1. **Защита данных** — никто не сможет «испортить» объект, минуя проверки.
- 2. **Свобода менять реализацию** — завтра можно заменить `_balance` на чтение из базы данных, а весь внешний код продолжит работать без изменений.
- 3. **Упрощение использования** — пользователю класса не нужно знать внутренности: достаточно публичного интерфейса.

Принцип 3. НАСЛЕДОВАНИЕ (Inheritance)

 **Точное определение:** наследование — механизм создания нового класса (наследника) на основе существующего (базового): наследник автоматически получает члены базового

класса и может **добавлять новые**, а также **изменять** поведение унаследованных виртуальных методов. Наследование выражает отношение «**является**» (**is-a**).

💡 **Аналогия:** «Собака **является** животным» — наследование уместно. «Машина **является** двигателем»? Нет! Машина **содержит** двигатель (отношение «имеет», *has-a*) — тут наследование будет ошибкой проектирования (см. ниже).

```
// БАЗОВЫЙ класс — общее для всех
class Animal
{
    public string Name { get; set; }

    public virtual void MakeSound()    // virtual: «наследникам РАЗРЕШЕНО переопределить»
    {
        Console.WriteLine($"{Name} издаёт звук");
    }

    public void Eat()                  // без virtual: метод общий для всех, менять нельзя
    {
        Console.WriteLine($"{Name} ест");
    }
}

class Dog : Animal    // «Dog является Animal» — двоеточие = наследование
{
    public void Fetch()                // СВОЁ новое поведение
        => Console.WriteLine($"{Name} приносит мяч");

    public override void MakeSound()   // переопределяем поведение родителя
        => Console.WriteLine($"{Name} говорит: Гав!");
}

class Cat : Animal
{
    public override void MakeSound()
        => Console.WriteLine($"{Name} говорит: Мяу!");
}

// Использование:
Dog dog = new Dog { Name = "Рекс" };
dog.MakeSound();    // Рекс говорит: Гав!    ← СВОЯ реализация (override)
```

```
dog.Eat();           // Рекс ест           ← УНАСЛЕДОВАНО от Animal
dog.Fetch();         // Рекс приносит мяч  ← ДОБАВЛЕНО в наследнике
```

Ключевые слова наследования:

Ключевое слово	Значение
<code>class Dog : Animal</code>	Dog наследуется от Animal
<code>virtual</code>	Метод родителя разрешает переопределение
<code>override</code>	Наследник переопределяет виртуальный метод
<code>abstract</code>	Метод без тела : родитель объявляет, наследники обязаны реализовать
<code>sealed</code>	Запрет: дальше наследоваться нельзя
<code>base.MakeSound()</code>	Вызов родительской реализации (удобно расширять, а не заменять)

Правила, которые важно знать:

- В С# наследование **одиночное**: у класса может быть только **один** прямой родитель (но интерфейсов можно реализовать сколько угодно — о них дальше в курсе).
- Наследуйтесь **только** при смысловом отношении «является».
- Не путайте «является» (is-a → наследование) и «имеет» (has-a → **композиция**: класс содержит объект другого класса как поле). Car **имеет** Engine :

```
class Engine
{
    public int Power { get; set; }
    public void Start() { /* ... */ }
}
```

```
class Car
{
    private readonly Engine _engine = new Engine(); // КОМПОЗИЦИЯ: содержит
    public void Start() => _engine.Start();
}
// Писать «class Car : Engine» — грубая ошибка: машина НЕ является двигателем.
```

Принцип 4. ПОЛИМОРФИЗМ (Polymorphism)

 **Точное определение:** полиморфизм — возможность объектов разных классов, имеющих общий базовый тип или интерфейс, **по-разному выполнять один и тот же вызов**. Код работает с общим типом, а конкретная реализация выбирается **во время выполнения** по фактическому типу объекта.

 **Простыми словами:** одна и та же команда — разная реакция. Скажите «издай звук» собаке, коту и корове — каждый поймёт по-своему. Ваш код команды при этом **не меняется**.

 **Аналогия:** кнопка «Сломать» на коробке с игрушками. У каждой игрушки внутри — свой механизм реакции: одна пицтит, другая разлетается. Кнопка одна — поведение разное.

Магия полиморфизма — один цикл, разное поведение:

```
List<Animal> zoo = new List<Animal>    // список БАЗОВОГО типа:
{                                       // в него кладутся любые наследники
    new Dog { Name = "Рекс" },
    new Cat { Name = "Мурка" }
};

foreach (Animal animal in zoo)    // компилятор видит только Animal
{
    animal.MakeSound();           // ...но вызывается метод РЕАЛЬНОГО объекта!
}
// Рекс говорит: Гав!
// Мурка говорит: Мяу!
```

Как это работает: компилятор проверяет, что у типа `Animal` **есть** метод `MakeSound` (безопасность), а решение «чей именно метод вызвать» принимается **во время выполнения** по фактическому типу объекта — это называется **динамическим (поздним) связыванием**.

Сила полиморфизма — расширение без изменения старого кода:

```
class Cow : Animal
{
    public override void MakeSound() => Console.WriteLine($"{Name}: Myy!");
}
```

```
// Добавили корову. Цикл foreach выше менять НЕ НУЖНО –
// новая корова «встроится» в существующий код автоматически.
```

Полиморфизм через интерфейс — пример из бизнеса:

```
class OrderService
{
    // Метод работает с АБСТРАКЦИЕЙ IPaymentMethod, не зная деталей
    public void Checkout(IPaymentMethod method, decimal total)
    {
        if (method.Pay(total))
            Console.WriteLine("Заказ оплачен");
    }
}

var service = new OrderService();
service.Checkout(new CreditCardPayment(), 1500);
service.Checkout(new CryptoPayment(), 1500);
// Завтра добавим ApplePayPayment – OrderService менять не придётся вообще.
```

Три вида полиморфизма в C#:

Вид	Механизм	Когда выбирается реализация
Динамический	virtual / override , интерфейсы	Во время выполнения (по фактическому типу)
Статический	Перегрузка методов: одно имя, разные параметры	Во время компиляции
Обобщения	List<T> , Dictionary<K,V>	Во время компиляции

```
// Статический полиморфизм — ПЕРЕГРУЗКА: имя одно, сигнатуры разные
class Printer
{
    public void Print(int value)      => Console.WriteLine($"Число: {value}");
    public void Print(string value) => Console.WriteLine($"Строка: {value}");
    public void Print(decimal v)     => Console.WriteLine($"Сумма: {v} руб.");
}

// Компилятор сам выберет нужный метод по типу аргумента:
new Printer().Print(42);           // Число: 42
new Printer().Print("привет");    // Строка: привет
```

4.5. Как все принципы работают вместе: сквозной пример

```
// АБСТРАКЦИЯ: объявляем ЧТО умеет любая фигура (но не КАК)
abstract class Shape
{
    public string Name { get; init; }    // init: задаётся только при создании

    public abstract double GetArea();    // каждая фигура считает по-своему
    public abstract double GetPerimeter();

    // Полиморфное поведение: внутри виртуального метода вызываются
    // абстрактные методы — конкретные версии подставляются в момент вызова
    public virtual string Describe() =>
        $"{{Name}}: S = {{GetArea():F2}}, P = {{GetPerimeter():F2}}";
}

// НАСЛЕДОВАНИЕ: конкретные фигуры строятся на базе абстракции
class Circle : Shape
{
    public double Radius { get; }

    public Circle(double r) { Name = "Круг"; Radius = r; }

    public override double GetArea()      => Math.PI * Radius * Radius;
    public override double GetPerimeter() => 2 * Math.PI * Radius;
}

class Rectangle : Shape
{
    public double Width { get; }
    public double Height { get; }

    public Rectangle(double w, double h) { Name = "Прямоугольник"; Width = w; Height = h; }

    public override double GetArea()      => Width * Height;
    public override double GetPerimeter() => 2 * (Width + Height);

    public override string Describe()      // расширяем родительское поведение
        => base.Describe() + " [прямоугольный]";
}

class Program
```

```
{
    static void Main()
    {
        // ПОЛИМОРФИЗМ: единый список разнородных фигур
        List<Shape> shapes = new()
        {
            new Circle(5),
            new Rectangle(3, 4)
        };

        foreach (Shape s in shapes)
            Console.WriteLine(s.Describe());
        // Круг: S = 78,54, P = 31,42
        // Прямоугольник: S = 12,00, P = 14,00 [прямоугольный]

        // LINQ + полиморфизм: суммарная площадь любых фигур
        Console.WriteLine($"Итого: {shapes.Sum(s => s.GetArea()):F2}"); // 90,54
    }
}
```

☞ Где здесь каждый принцип:

- **Абстракция** — *Shape описывает понятие «фигура» без деталей;*
- **Инкапсуляция** — *данные (Radius , Width) доступны через свойства; изменение состояния контролируется классом;*
- **Наследование** — *Circle и Rectangle переиспользуют и развивают Shape ;*
- **Полиморфизм** — *один цикл корректно работает с любыми фигурами, включая те, что будут написаны в будущем.*

4.6. Типичные ошибки новичков

Ошибка	Почему плохо	Как правильно
Публичные поля вместо свойств	Нет контроля данных, нельзя расширять	Поля private + свойства с логикой
Наследование «ради переиспользования кода»	Хрупкая иерархия: изменение родителя ломает детей	Наследовать только при отношении «является»; иначе — композиция

Ошибка	Почему плохо	Как правильно
Цепочки наследования $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$	Изменение корня ломает всё	Не глубже 2–3 уровней, предпочитайте композицию
Возвращать из метода внутреннюю изменяемую коллекцию напрямую	Снаружи её изменят — инкапсуляция нарушена	Возвращать копию или read-only обёртку
Класс-«бог» на 2000 строк	Нечитаемо, нетестируемо	Разделять по ответственности

4.7. Выводы

Принцип	Суть одной фразой	Инструменты C#
Абстракция	Показываем «что», скрываем «как»	Интерфейсы, абстрактные классы
Инкапсуляция	Данные спрятаны, доступ — только по правилам класса	<code>private</code> , свойства, методы
Наследование	«Является»: переиспользуем и расширяем базовый класс	<code>:</code> , <code>virtual</code> , <code>override</code> , <code>base</code>
Полиморфизм	Один вызов — разные реализации по фактическому типу	<code>virtual/override</code> , интерфейсы, перегрузка, <code>generics</code>

ООП — не догма, а инструмент **моделирования**: он позволяет писать код, который (1) читается как описание предметной области, (2) защищён от некорректного использования, (3) расширяется без переделки старого. Эти свойства — основа устойчивых больших систем.

Контрольные вопросы:

1. В чём разница между полем и свойством? Почему публичные поля считаются плохим стилем?
2. Объясните, как в примере с банкоматом работают все элементы: хранилище, кнопки, проверка PIN.
3. Чем «является» отличается от «имеет»? Почему `class Car : Engine` — ошибка?
4. Чем `virtual` отличается от `abstract` ? Что произойдёт, если наследник не реализует абстрактный метод?

5. Когда выбирается реализация метода при динамическом полиморфизме — при компиляции или при выполнении? Почему это важно?
6. Пусть в сквозной пример добавят `class Triangle : Shape`. Что нужно изменить в остальном коде, чтобы треугольники обрабатывались? Обоснуйте ответ.

Практическое задание: Спроектируйте систему «Библиотека»:

1. Абстрактный класс `LibraryItem`: приватное поле названия (свойство с проверкой на пустоту), инвентарный номер (только чтение), абстрактный метод `GetInfo()`.
2. Наследники `Book` (автор, число страниц), `Magazine` (номер выпуска), `DVD` (длительность) — каждый со своей реализацией `GetInfo()`.
3. Продемонстрируйте полиморфизм: список `List<LibraryItem>` и цикл вывода информации.
4. *Со звёздочкой*: добавьте интерфейс `IBorrowable { void Borrow(); }` — только книги и DVD можно выдать на руки; выведите в цикле только «выдаваемые» экземпляры.

Рекомендуемые ресурсы

1. **Официальная документация Microsoft** — learn.microsoft.com/dotnet (первоисточник, есть на русском).
2. Э. Троелсен, Ф. Джепикс — «С# 10 и платформа .NET 6» (систематический учебник).
3. Дж. Рихтер — «CLR via C#» (глубокое устройство платформы — к лекциям 3–4, после освоения основ).
4. Р. Мартин — «Чистый код» (принципы проектирования — к лекции 4).
5. metanit.com/sharp — русскоязычный справочник с примерами.

 **Примечание:** сведения о версиях .NET и сроках поддержки соответствуют моменту составления курса; актуальные данные проверяйте на dotnet.microsoft.com.